

RAG Pipeline - Master Architecture Guide

This document serves as the master map for the RAG pipeline, categorizing every technique from basic indexing to advanced agentic loops.

1. The Data Layer: Indexing Strategies

Goal: How do we store the data so it can be found?

Strategy	Best For	Mechanism
Standard Indexing	General purpose.	Chunk → Embed → Store.
Multi-Representation	"Needle in a haystack."	Embeds a summary/proposition; retrieves the full parent document.
RAPTOR	Thematic/Abstract queries.	Recursively clusters and summarizes chunks into a hierarchical tree.
ColBERT	Fine-grained precision.	Stores vector-per-token (Late Interaction) to match exact terminology/phrasing.

2. The Core Execution: Retrieval & Generation

Goal: The basic "R" and "G" steps.

- **Retrieval:** Fetching the relevant context using Dense (Vector), Sparse (Keyword), or Hybrid search.
- **Generation:** Synthesizing the answer using the retrieved context and a grounded prompt.

3. The Pre-Retrieval Layer: Query Intelligence

Goal: Optimizing the user's input before searching.

A. Query Translation (Refining the Question)

- **Multi-Query:** Generates different perspectives of the same question to broaden recall.
- **RAG Fusion:** Generates multiple queries + Re-ranks the results using RRF (Reciprocal Rank Fusion).
- **Decomposition:** Breaks complex questions into simpler sub-queries.
- **Step-Back:** Generates a broader, abstract question to find background context.
- **HyDE:** Generates a fake "hypothetical answer" to match against real documents.

B. Routing (Directing Traffic)

- **Logical Routing:** Uses an LLM to reason and pick the best data source.
- **Semantic Routing:** Uses embedding similarity to pick the best route (faster).

C. Query Construction (Speaking the Database's Language)

- **Text-to-SQL:** Converts natural language to SQL for relational DBs.
- **Self-Querying:** Converts natural language to Metadata Filters for Vector DBs.

4. The Agentic Layer: Architectures

Goal: Putting it all together into a "Thinking" system.

A. Corrective RAG (CRAG)

- **Philosophy:** "Trust but verify."
- **Mechanism:** Retrieves documents → **Grades** them.
 - If Correct → Refine (Knowledge Strips) → Generate.
 - If Incorrect/Ambiguous → **Web Search** fallback.

B. Adaptive RAG

- **Philosophy:** "Route by complexity."
- **Mechanism: Classifies** the query difficulty first.
 - Simple → No Retrieval.
 - Moderate → Standard RAG.
 - Complex → Iterative Agent Loop.

C. Self-RAG (Self-Reflective RAG)

- **Philosophy:** "Self-Correction during generation."
- **Mechanism:** It doesn't just grade the *documents* (like CRAG); it grades the *generation* itself using special "Reflection Tokens."
 1. **Retrieve:** Decides *if* it needs retrieval.
 2. **Generate:** Generates a segment.
 3. **Critique:** Self-evaluates: "Is this supported by the text?" and "Is this useful?"
 4. **Refine:** If the critique is bad, it regenerates the answer automatically.